# Formal Languages and Compilers
# Prof. Breveglieri, Morzenti, Agosta
# Written exam: laboratory question 08/06/2023

**Time: 60 minutes.** Textbooks and notes can be used. Pencil writing is allowed.
**Important:** Write your name on any additional sheet.

SURNAME (Cognome): .....

NAME (Nome): .....

Matricola: ..... or Person Code: .....

Instructor: ☐ Prof. Breveglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the `Acse` compiler given with the exam text.

Modify the specification of the lexical analyser (`flex` input) and the syntactic analyser (`bison` input) and any other source file required to extend the `Lance` language with the ability to search an array for elements with a given value, and then reassign them with another value.

This operation is performed using a new statement called **`replace`**. The syntax of the statement is the following:

**`replace`**`(`⟨*array*⟩`,` ⟨*exp. 1*⟩`,` ⟨*exp. 2*⟩`);`

The first argument, ⟨*array*⟩, is the identifier of the array being modified by the statement. The second argument, ⟨*exp. 1*⟩, is the expression whose value must be searched in the array. The third and last argument, ⟨*exp. 2*⟩, is the new value that will be assigned to all items found with the same value as ⟨*exp. 1*⟩. If no elements of the array have the same value of ⟨*exp. 1*⟩, the array is left unchanged. Additionally, if ⟨*array*⟩ is not a valid array identifier, a syntax error is raised at compile-time.

The following code snippet exemplifies the use of the statement. Initially, we assume the contents of array `a` are initialized to the integer numbers from 1 to 10 (the code to initialize `a` is not shown for brevity). The first use of `replace` replaces all items in `a` whose value is $b - 17 = 7$, with 5. Only the item at index 6 is equal to 7, therefore its value is replaced with 5. The second use of `replace` replaces all items whose value is 5 with $\frac{b}{2} = 12$. The items at index 4 and 6 satisfy the condition and are indeed replaced with the value 12. Finally, the third `replace` finds all items of value $-b \times 10 = -120$ and replaces them with $-24 + b = 0$. Since there are no items in the array with the specified value, the array stays unchanged.

```
int a[10], b=24;

/* a == {1, 2, 3, 4, 5, 6, 7, 8, 9,10}; */
replace(a, b-17, 5);
/* a == {1, 2, 3, 4, 5, 6, 5, 8, 9,10}; */
replace(a, 5, b/2);
/* a == {1, 2, 3, 4,12, 6,12, 8, 9,10}; */
replace(a, (-b)*10, -24+b);
/* a == {1, 2, 3, 4,12, 6,12, 8, 9,10}; */
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (2 points)
2. Define the syntactic rules or the modifications required to the existing ones. (3 points)
3. Define the semantic actions needed to implement the required functionality. (20 points)

## Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '−' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..e3361a0 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,7 @@ ID       [a-zA-Z_][a-zA-Z0-9_]*
 "return"          { return RETURN; }
 "read"            { return READ; }
 "write"           { return WRITE; }
+"replace"         { return REPLACE; }

 {ID}              { yylval.svalue=strdup(yytext); return IDENTIFIER; }
 {DIGIT}+          { yylval.intval = atoi( yytext );
diff --git a/acse/Acse.y b/acse/Acse.y
index 0636dc6..e89a401 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -125,6 +125,7 @@ extern void yyerror(const char*);
 %token RETURN
 %token READ
 %token WRITE
+%token REPLACE

 %token <label> DO
 %token <while_stmt> WHILE
@@ -247,6 +248,7 @@ statements  : statements statement       { /* does nothing */ }
 statement   : assign_statement SEMI      { /* does nothing */ }
             | control_statement          { /* does nothing */ }
             | read_write_statement SEMI  { /* does nothing */ }
+            | replace_statement SEMI
             | SEMI            { gen_nop_instruction(program); }
 ;

@@ -260,6 +262,53 @@ read_write_statement : read_statement  { /* does nothing */ }
                      | write_statement { /* does nothing */ }
 ;

+replace_statement: REPLACE LPAR IDENTIFIER COMMA exp COMMA exp RPAR
+   {
+      /* Check if the identifier exists and that it belongs to an array. */
+      t_axe_variable *v_array = getVariable(program, $3);
+      if (!v_array || !v_array->isArray) {
+         yyerror("first argument must be an array");
+         YYERROR;
+      }
+
+      /* Begin generating a do-while loop which will iterate through all the
+       * elements of the array. Reserve a register to use as a loop counter. */
+      int r_i = gen_load_immediate(program, 0);
+      /* Generate a label at the beginning of the loop body. */
+      t_axe_label *l_loop = assignNewLabel(program);
+
+      /* Generate code to load the current array element and compare it with
+       * the first expression. */
+      int r_val = loadArrayElement(program, $3, create_expression(r_i, REGISTER));
+      if ($5.expression_type == IMMEDIATE) {
```

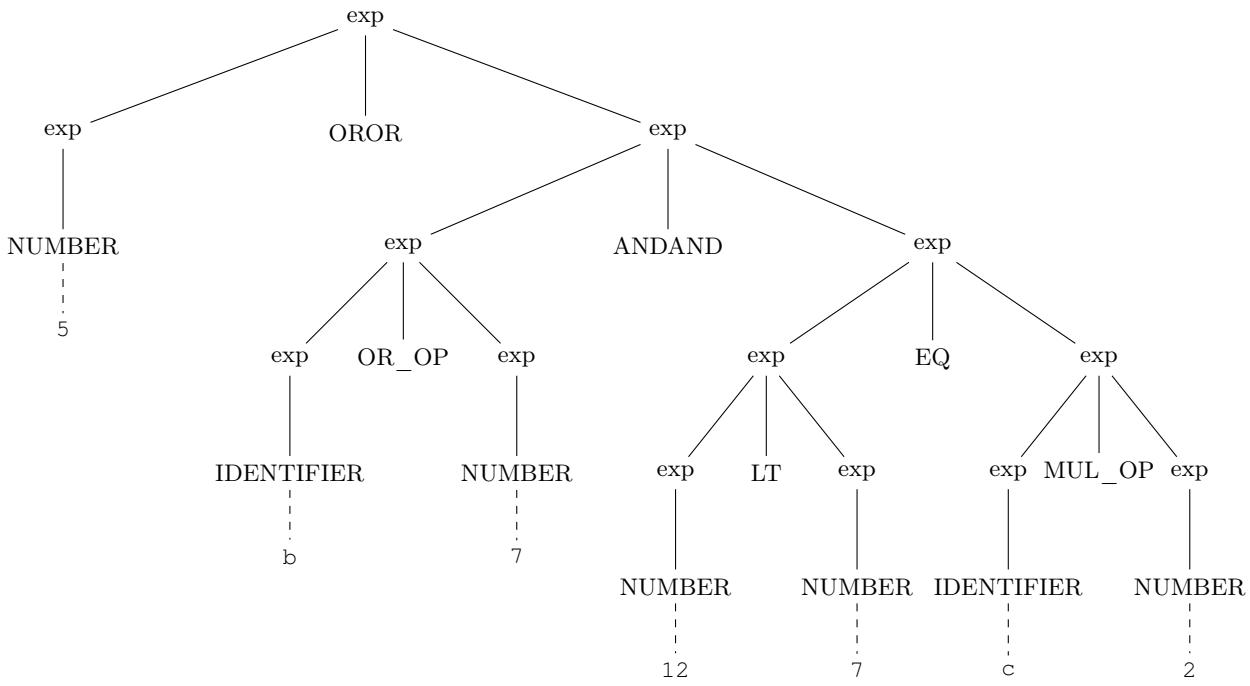
```
+         gen_subi_instruction(program, REG_0, r_val, $5.value);
+      } else {
+         gen_sub_instruction(program, REG_0, r_val, $5.value, CG_DIRECT_ALL);
+      }
+      /* Generate a branch that will skip over the assignment of the new value
+       * if the array element is different than the expression, and therefore
+       * shall not be modified. */
+      t_axe_variable *l_skipAssign = newLabel(program);
+      gen_bne_instruction(program, l_skipAssign, 0);
+      /* Generate the instructions that will overwrite the current array
+       * element's value with the new one when the comparison is successful. */
+      storeArrayElement(program, $3, create_expression(r_i, REGISTER), $7);
+      /* Generate the label used before to skip the assignment. */
+      assignLabel(program, l_skipAssign);
+
+      /* Update the loop counter and compare it with the length of the array. */
+      gen_addi_instruction(program, r_i, r_i, 1);
+      gen_subi_instruction(program, REG_0, r_i, v_array->arraySize);
+      /* If the loop counter is less than the length of the array, jump back
+       * to perform a new iteration. */
+      gen_blt_instruction(program, l_loop, 0);
+
+      /* Free the identifier because it was allocated dynamically by the rules
+       * in Acse.lex. */
+      free($3);
+   }
+;
+
 assign_statement : IDENTIFIER LSQUARE exp RSQUARE ASSIGN exp
                {
                   /* Notify to 'program' that the value $6
diff --git a/tests/replace/replace.src b/tests/replace/replace.src
new file mode 100644
index 0000000..3ff913e
--- /dev/null
+++ b/tests/replace/replace.src
@@ -0,0 +1,19 @@
+int a[10], b=24;
+int i;
+
+i = 0;
+while (i < 10) {
+   a[i] = i + 1;
+   i = i + 1;
+}
+
+replace(a, b-17, 5);
+replace(a, 5, b/2);
+replace(a, (-b)*10, -24+b);
+
+/* a == {1, 2, 3, 4,12, 6,12, 8, 9,10}; */
+i = 0;
+while (i < 10) {
+   write(a[i]);
+   i = i + 1;
+}
```

4. Given the following `Lance` code snippet:

```
5 || b | 7 && 12 < 7 == c * 2
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the* `exp` *nonterminal.* (5 points)

## Solution



5. (**Bonus**) Briefly explain in plain English words how to modify the solution given to the questions about the `replace` statement in order to add support for performing multiple replacements in sequence, like in the following example.

```
int a[10];

/* a == {1, 2, 3, 4, 5, 6, 7, 8, 9,10}; */
replace(a, 1, 2, 2, 3); // Replace 1 with 2, then 2 with 3
/* a == {3, 3, 3, 4, 5, 6, 7, 8, 9,10}; */
replace(a, 10, 9, 8, 7, 6, 5); // 10->9, 8->7, 6->5
/* a == {3, 3, 3, 4, 5, 5, 7, 7, 9, 9}; */
```